

Relazione-3

Directory cpi: (Calcolo di pi-greco)

cpi.c:

Usando prima il codice cpi.c(senza **Parallelizzazione**) Ecco I risultati al variare n (il numero di intervalli).

Output di cpi.c:

```
root@wn1:~/3-Progettazione/Progettazione/cpi# ./cpi
#Inter      error      time      hostname
 1000000000, 1.77635684e-13, 2.32333, wn1
root@wn1:~/3-Progettazione/Progettazione/cpi# ./cpi -n 1
#Inter      error      time      hostname
 1, 5.84073464e-02, 0.00000, wn1
root@wn1:~/3-Progettazione/Progettazione/cpi# ./cpi -n 2
#Inter      error      time      hostname
 2, 2.07602876e-02, 0.00000, wn1
root@wn1:~/3-Progettazione/Progettazione/cpi# ./cpi -n 3
#Inter      error      time      hostname
 3, 9.25655628e-03, 0.00000, wn1
root@wn1:~/3-Progettazione/Progettazione/cpi# |
```

What we see is that increasing the number of interval reduces the time it takes for calculation and also reduces the error.

cpi+amdhal.c:

We have to know that amdahl defines the total execution time of a program as the sum of the time of execution of the part not parallel and the time of the part parallel. That is:

$$T_s = T_{np} + T_p$$

Parallelism of the code is done by adding a sleep on the sequential time calculated when the parallel part starts its execution. This is a screenshot of the part here.

```
usleep(s); // Simulazione codice non parallelizzabile
clock_gettime( CLOCK_REALTIME , &tb); //////////////////////////////////
//      sum= f1(n);
//      sum= f2(n);
clock_gettime( CLOCK_REALTIME , &tc); //////////////////////////////////
```

This is the result of the cpi+Amdahl:

```
root@wn1:~/3-Progettazione/Progettazione/cpi# ./cpi+amdahl
#Inter      error      tp      tnp      tp1+tnp      hostname
100000000,  6.33271213e-13,  0.24312,  0.10061,  0.34372,  wn1
root@wn1:~/3-Progettazione/Progettazione/cpi#
```

We can clearly see that the the total time that is $tp + tnp$ is less that that of `cpi.c` which is all serial.

Now I added the result I got in the python code to get the .png this is where I added my values

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

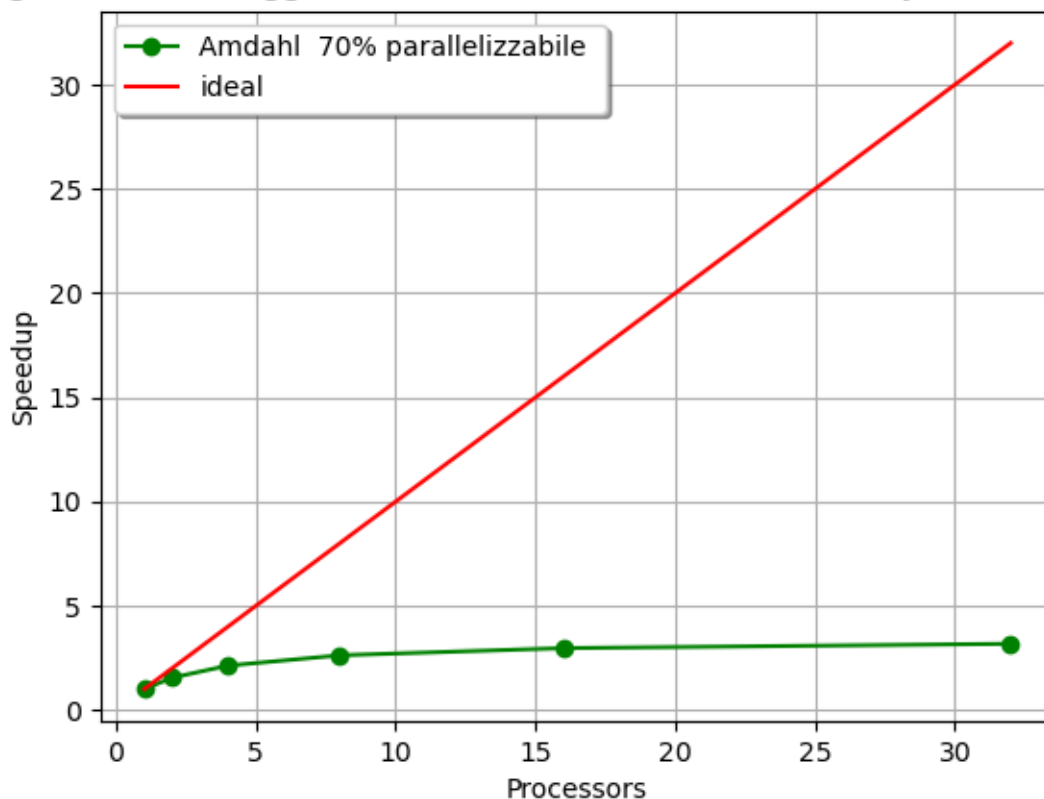
tnp=0.10061
tp=0.24312

Qp=int(100*tp/(tp+tnp))
etichetta="Amdahl  %d%% parallelizzabile " % (Qp)

N=[1,2,4,8,16,32] # numero processori
Samdahl=[(tnp+tp)/(tnp+tp/n) for n in N]

plt.title('Progettazione - "Legge di Amdahl"  Nna Minkouse Kenneth James 11/03/2026')
plt.grid()
plt.xlabel('Processors')
plt.ylabel('Speedup')
```

Progettazione - "Legge di Amdahl" Nna Minkouse Kenneth James 11/03/202



HEAT

- Sulla heat ho fatto la run del programma heat.c con il comando:
`gcc -O2 heat.c -o heat`
- Ho fatto generare il heatmap e ecco il risultato :

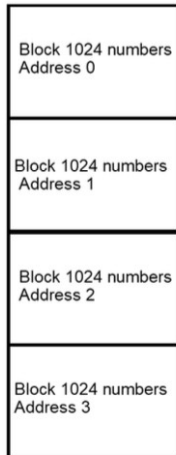


- Si vede che I bordi e il cento hanno la temperatura costante.
-

Esercizio Fattorizzazione

Example:

Modulus 24 bit (2^{24} numbers)
Primes to test 12 bit (2^{12} numbers)
Block 10 bit size 1024
Address 2 bit size 4



We know that in this problem we want to use brute force to obtain the modulus that is part of a private and public key.(the algorithm RSA of openssl uses modulus for encryption and is someone finds that the 2 big number that when multiply gives the modulus has cracked the keys).

$$\Rightarrow \text{modulus } (n) = p * q$$

→ first we generate a modulus using openssl with the algorithm RSA

openssl genrsa -out rsa_key.pem 512

openssl rsa -in rsa_key.pem -modulus -noout

This is the modulus generated:

```
root@wn1:~/3-Progettazione/Progettazione/factorize# openssl genrsa -out rsa_key.pem 512
root@wn1:~/3-Progettazione/Progettazione/factorize# openssl rsa -in rsa_key.pem -modulus -noout
Modulus=DB9E9F69063689A0BE381EA264ACE549705BA59F0CBAB97E384E93149B6C8B3DB09BF76A62CB8F9FB02524BB36D980252F9150B8E3C1B9A29E0096D3606AB799
root@wn1:~/3-Progettazione/Progettazione/factorize#
```

But as we want to generate a small sample for testing if the code fatoriza.c can actually find the p and q we are going to use a small modulus compiling the following executable.

./factorize -g 68

generating modulus of 68 bits

```
root@wn1:~/3-Progettazione/Progettazione/factorize# ./factorize -g 68
generating modulus of 68 bits
P=038B1203C7 Q=031FF8155F
Modulus=0B127C3EF6D7C6B9D9
root@wn1:~/3-Progettazione/Progettazione/factorize# |
```

using **4 bit** to represent 1 block address(index) it means in total we are going to have **$2^4 = 16$ blocks** (so that is the numbers will be divided in to 16 blocks)

Since the numbers are very big it will take a longer time because we are having less range. Look the diagram below we can se that the time it takes is very large thats 354.2 sec etc just for one block

```
# Modulus: B127C3EF6D7C6B9D9 72 bits - prime: address 4 bit, block 32 bit

processing block 15
processed block 15 in 354.2 sec. with result 0
processing block 14
processed block 14 in 359.9 sec. with result 0
processing block 13
processed block 13 in 367.1 sec. with result 0
processing block 12
|
```

Looking for p and q using address block of 10bit that means $2^{10} = 1024$ blocks , the time is less per block look that the picture below.

```
root@wn1:~/3-Progettazione/Progettazione/factorize# ./factorize -m 0B127C3EF6D7C6B9D9 -a 10

# Modulus: B127C3EF6D7C6B9D9 72 bits - prime: address 10 bit, block 26 bit

processing block 1023
processed block 1023 in 5.3 sec. with result 0
processing block 1022
processed block 1022 in 5.2 sec. with result 0
processing block 1021
```

We see that it takes us 5.3 sec for a block much more faster that 16 blocks with very large numbers in.